

Informazione



Contrassegna
domanda

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro il 29/06/2025, alle ore 23:59, mediante caricamento su Portale. Per i laureandi che intendano presentare domande di laurea nella sessione di luglio 2025, la scadenza è il 27/6 alle 23:59.
- assicurarsi di caricare l'elaborato nella **Sezione Elaborati relativa all'a.a. 2024/25**.
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale
- QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE. Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Domanda 1

Completo

Non valutata



Contrassegna
domanda

Indicare quale tipologia di esame si intende svolgere, selezionando UNA sola scelta tra a (12 punti) e b (18 punti).

E' possibile, in aggiunta, un'ulteriore opzione c, da selezionare SOLO nel caso di laureando/a che intenda (e possa) presentare domanda di laurea per la sessione di LUGLIO 2025.

Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo eventuale orale, CARICARE LA RELAZIONE NEGLI ELABORATI entro il 27/6 alle 23:59.

Scegli una o più alternative:

- ☒ a. 12 Punti - Semplificato
- ☐ b. 18 Punti - Completo
- ☐ c. Selezionare SOLO nel caso di laureando/a che intenda (qualora si superi l'esame) presentare domanda di laurea nella sessione di LUGLIO 2025.
Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo orale. CARICARE LA RELAZIONE entro il 27/6 alle 23:59.

Risposta parzialmente esatta.

Ignorare il feedback di questa domanda.

Le risposte corrette sono: 12 Punti - Semplificato, 18 Punti - Completo, Selezionare SOLO nel caso di laureando/a che intenda (qualora si superi l'esame) presentare domanda di laurea nella sessione di LUGLIO 2025.

Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo orale. CARICARE LA RELAZIONE entro il 27/6 alle 23:59.

Domanda 2

Completo

Punteggio max:
3,00

Contrassegna domanda

È dato un ADT List, in grado di rappresentare una lista (non ordinata) di stringhe (allocate dinamicamente). Si completi la funzione listDeleteLonger che, data la lista e una lunghezza (un valore intero lmax), elimini dalla lista le stringhe (e i relativi nodi) di lunghezza superiore a lmax. La funzione ritorna un risultato intero, che rappresenta il numero di cancellazioni effettuate. Completare il codice parziale utilizzando la numerazione riportata

```
typedef struct node {
    char *val;
    struct node *next;
} link;
...
int listDeleteLonger
(List l, int lmax) {
link x, p;
int cnt=0;
for
(x=l->head, p=NULL; x!=NULL; <1> ) {
    if (strlen(<2>)) {
        <3> // bypass
        <4> // free
        <5> // other
    }
    else {
        <6>
    }
}
return cnt ;
}
```

<1>	
<2>	
<3>	
<4>	
<5>	
<6>	

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

<1>	T
<2>	T
<3>	T
<4>	T
<5>	T
<6>	T

Domanda 3

Completo

Punteggio max.:
4,00Contrassegna
domanda

È dato un BST (tipo ADT BST). Si scriva una funzione che duplichi il BST, cioè generi un secondo BST con lo stesso contenuto del primo. Si supponga di poter utilizzare una funzione

Item ItemDup(Item i);

per generare un duplicato (una copia) di un dato Item.

```
BST BstDup (BST b) {
```

```
}
```

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
//scrivere qui il codice... T
```

```
Item ItemDup(Item i);
```

```
BST BstDup (BST b) { }
```

Domanda 4

Completo

Punteggio max.:
5,00Contrassegna
domanda

È dato un Grafo orientato realizzato con liste adiacenza (tipo ADT Graph). Si scriva una funzione GRAPHcheckSimmetry che, ricevuto come parametro un grafo, verifichi se il grafo è simmetrico e non contiene cappi (self-loop). Si ricorda che un grafo si dice simmetrico quando per ogni arco $v-w$ esiste anche l'arco inverso $w-v$.

```
int GRAPHcheckSimmetry (Graph g) {  
  
  
}
```

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
int GRAPHcheckSimmetry (Graph g) {  
T  
  
}
```

InformazioneContrassegna
domanda**PAGINA DI INTERMEZZO**

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

Informazione



Contrassegna
domanda

Descrizione del problema

E' dato un insieme di N oggetti, ognuno caratterizzato da costo, peso e volume: un file testo contiene, per ogni oggetto, su una riga, 4 campi separati da spazi: il nome (stringa univoca, priva di spazi di massimo 30 caratteri), il costo, il peso e il volume (tre numeri reali).

E' dato un secondo file in cui sono riportati, uno per riga, dei vincoli tra coppie di oggetti: ogni riga del file contiene due nomi di oggetti vincolati fra loro (un oggetto può comparire nessuna o più volte nel file).

Occorre pianificare il trasporto degli oggetti, tenendo conto che è disponibile un solo mezzo di trasporto con capienza massima sia in peso (P_{max}) che in volume (V_{max}).

- Il mezzo può essere riempito una sola volta al giorno (il carico giornaliero), per il conseguente trasporto giornaliero: Il carico giornaliero non deve superare dati limiti di peso e volume
- Coppie di oggetti vincolati vanno trasportate insieme nello stesso carico giornaliero,

Si vuole organizzare la distribuzione degli oggetti in più carichi giornalieri, minimizzando il numero di carichi (cioè di giorni) necessari. A parità di giorni, si vuole minimizzare la differenza tra il costo complessivo massimo per un giorno e quello minimo (relativo a un altro giorno).

Domanda 5

Completo

Punteggio max.:
4,00



Contrassegna
domanda

Struttura dati

Definire e implementare delle opportune strutture dati per rappresentare, come ADT di prima classe:

- L'elenco degli oggetti (tipo OBJECTS)
- I vincoli (tipo CONSTRAINTS)
- una soluzione al problema di ottimizzazione (tipo SOL)

In aggiunta alle tre strutture dati esplicitamente richieste, è possibile definire tutti i tipi ausiliari ritenuti opportuni, a supporto delle tre strutture principali. NON è richiesta la funzione di lettura da file.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
//scrivere qui il codice...
```

T

Domanda 6

Completo

Punteggio max.:
6,00Contrassegna
domanda**Problema di verifica**

Un insieme di oggetti è valido se la capienza del mezzo di trasporto (volume e peso) è tale da poter contenere nello stesso carico tutti gli oggetti che, in base ai vincoli, debbono stare insieme. Si scriva una funzione di verifica che, dato un elenco di oggetti (tipo OBJECTS), un insieme di vincoli (tipo CONSTRAINTS), e due numeri indicanti peso a capienza massima, verifichi la validità.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
//scrivere qui il codice...
```

T

Domanda 7

Completo

Punteggio max.:
8,00Contrassegna
domanda**Problema di ricerca e ottimizzazione**

Scrivere una funzione ricorsiva in grado di individuare, a partire da un elenco di oggetti (tipo OBJECTS), un insieme di vincoli (tipo CONSTRAINTS), e due numeri indicanti peso a capienza massima, la distribuzione ottima degli oggetti in più carichi giornalieri. Si deve minimizzare il numero di carichi (cioè di giorni) necessari. A parità di giorni, si minimizzi la differenza tra il costo complessivo massimo per un giorno e quello minimo (relativo a un altro giorno). Il costo relativo a un carico/giorno è la sommatoria degli oggetti inseriti nel carico.

Il modello del calcolo combinatorio scelto va indicato e spiegato/motivato.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
// Indicare e motivare qui il modello di calcolo combinatorio usato e lo spazio in cui si cerca
```



T

```
//scrivere qui il codice...
```

T

TEORIA

Domanda 1

Completo

Punteggio max.:
2,50



Contrassegna
domanda

Si risolva la seguente equazione alle ricorrenze mediante il metodo dello sviluppo (unfolding):

$$\begin{array}{ll} T(n) = 4T(n/2) + n^2 & n > 1 \\ T(1) = 1 & n = 1 \end{array}$$

Formato della risposta: T

L'equazione rappresenta un algoritmo di tipo

- divide and conquer: sì/no
- decrease and conquer: sì/no

$T(n/2) =$

$T(n)$ dopo 3 passi di unfolding:

$T(n) =$

limite superiore della sommatoria =

$T(n) = O(\dots)$

Completo

Punteggio max.:
2,50

Contrassegna
domanda

Siano date 2 catene di $n=4$ stazioni di montaggio S_{ij} ciascuna ($0 \leq i \leq 1, 0 \leq j < 4$) con i seguenti dati:

- tempi di lavorazione a nella catena i alla stazione j

0	7	4	6	5
1	2	8	3	4
	0	1	2	3

- tempi di trasferimento da una catena all'altra dalla stazione $j-1$ alla stazione j (t nullo tra stazioni della stessa catena)

0	1	2	1
1	2	1	3
	0	1	2

- tempi di entrata e

2	4
0	1

- tempi di uscita x

2	1
0	1

matrice a : tempi di lavorazione (catena i , stazione j)

Mediante un algoritmo di programmazione dinamica si determini il costo minimo di lavorazione e la sequenza corrispondente di stazioni di montaggio. Si riportino le tabelle f ed l dei costi e dei passaggi.

Formato della risposta: T
contenuto delle matrici f e l

$f[0,0]=$

$f[0,1]=$

$f[0,2]=$

$f[0,3]=$

$f[0,4]=$

$f[1,0]=$

$f[1,1]=$

$f[1,2]=$

$f[1,3]=$

$f[1,4]=$

$l[0,0]=$

$l[0,1]=$

$l[0,2]=$

$l[0,3]=$

$l[0,4]=$

$l[1,0]=$

$l[1,1]=$

$l[1,2]=$

$l[1,3]=$

$l[1,4]=$

tempo minimo:

sequenza di stazioni di montaggio

entrata dalla catena 0/1

stazione 0: catena 0/1

stazione 1: catena 0/1

stazione 2: catena 0/1

stazione 3: catena 0/1

uscita dalla catena 0/1

Domanda 3

Completo

Punteggio max.:
1,00Contrassegna
domanda

Si converta la seguente espressione da forma infissa a forma prefissa:

$$(2 \times 3 + 4/5) \times 8 - 4$$

Formato: sulla stessa riga elenco di operandi e operatori separati da virgole o spazi. T

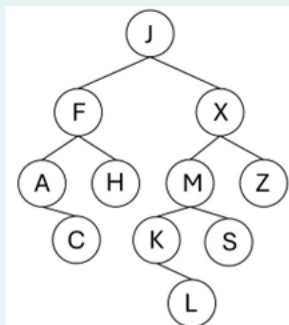
Espressione in forma prefissa:

Domanda 4

Completo

Punteggio max.:
2,00Contrassegna
domanda

Si partizioni il seguente BST attorno alla chiave mediana. In ogni nodo compare la chiave carattere, altre eventuali informazioni non sono riportate.



Quale è la chiave mediana?

Quale informazione contiene il nodo con chiave F oltre alla chiave stessa?

Quale informazione contiene il nodo con chiave J oltre alla chiave stessa?

Dopo il partizionamento, chi sono i figli sinistro e destro della radice?

formato della risposta: chiave, informazione supplementare

figlio SX=

figlio DX =

Dopo il partizionamento, chi sono i figli sinistro e destro di J?

formato della risposta: chiave, informazione supplementare

figlio SX=

figlio DX =

Dopo il partizionamento, chi sono i figli sinistro e destro di M?

formato della risposta: chiave, informazione supplementare

figlio SX=

figlio DX =

Domanda 5

Completo

Punteggio max.:
2,00Contrassegna
domanda

Si inseriscano in sequenza le $N=7$ chiavi intere

138 48 103 60 213 28 98

in una tabella di hash (inizialmente vuota) gestita con open addressing con double hashing. Si determini la dimensione minima M della tabella di per avere un fattore di carico $\alpha < 0.4$.

Formato della risposta: T

dimensione minima della tabella

$M =$

Si esprima la funzione di hash utilizzata:

$h(k, i) =$

Collisioni chiave 98:

cella_1 dove vi è collisione:

...

cella_n dove vi è collisione:

Al termine dell'inserzione, in quale cella si trova la chiave 213?

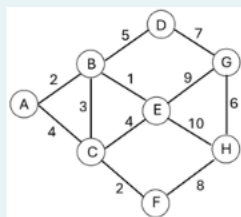
Al termine dell'inserzione, cosa contiene la cella all'indice 13?

Domanda 6

Completo

Punteggio max:
2,00Contrassegna
domanda

Dato il seguente grafo non orientato, connesso e pesato, se ne determini un minimum spanning tree applicando l'algoritmo di Prim a partire dal vertice **A**, ritornando come risultato il valore del peso minimo e gli archi selezionati. Si esplicitino i passi intermedi.



Formato della risposta: T

Configurazione al I taglio:

S= (elenco vertici)

V-S = (elenco vertici)

Configurazione al II taglio:

S= (elenco vertici)

V-S = (elenco vertici)

Configurazione al III taglio:

S= (elenco vertici)

V-S = (elenco vertici)

Configurazione al IV taglio:

S= (elenco vertici)

V-S = (elenco vertici)

Peso minimo:

Gli archi seguenti appartengono ad un MST:

A-B: sì/no

A-C: sì/no

B-C: sì/no

B-D: sì/no

B-E: sì/no

C-E: sì/no

C-F: sì/no

D-G: sì/no

E-G: sì/no

E-H: sì/no

F-H: sì/no

G-H: sì/no